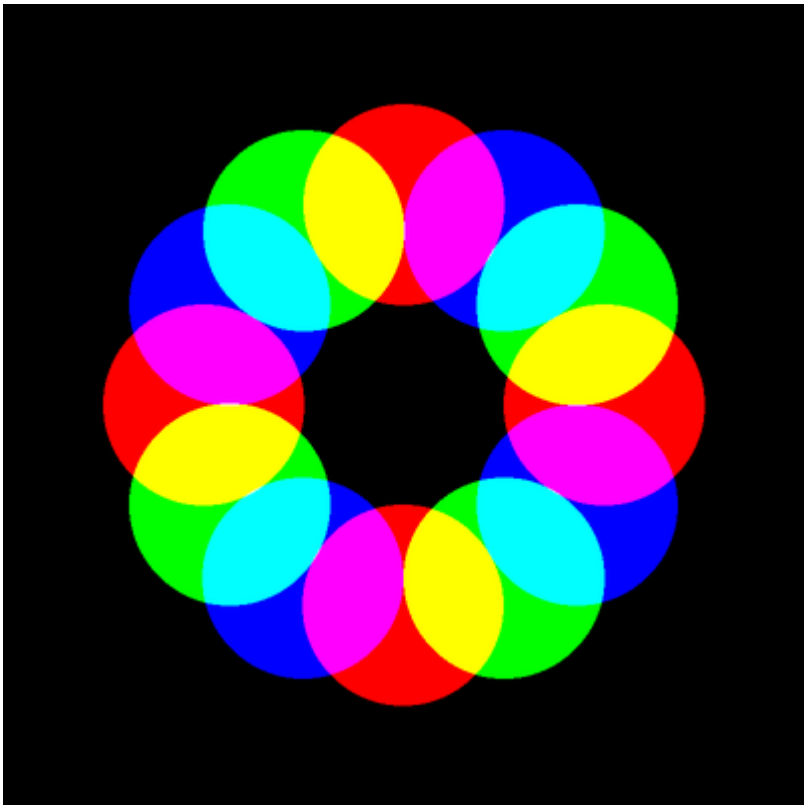
Cours 13 — Trigonométrie : des cercles sur un cercle

> **Avant** : cours 07 (cosinus pour osciller), cours 09 (la roue des teintes). **Comment travailler** : toujours dans `ofApp.h` et `ofApp.cpp`, par versions successives. `ofApp13.h` / `ofApp13.cpp` : la référence téléchargeable de l'état final.

Douze cercles disposés régulièrement sur un cercle invisible, qui se rapprochent et s'éloignent du centre. Pour placer un point « à tel angle, à telle distance », il faut `cos` et `sin` — c'est le premier des quatre cours de trigonométrie.



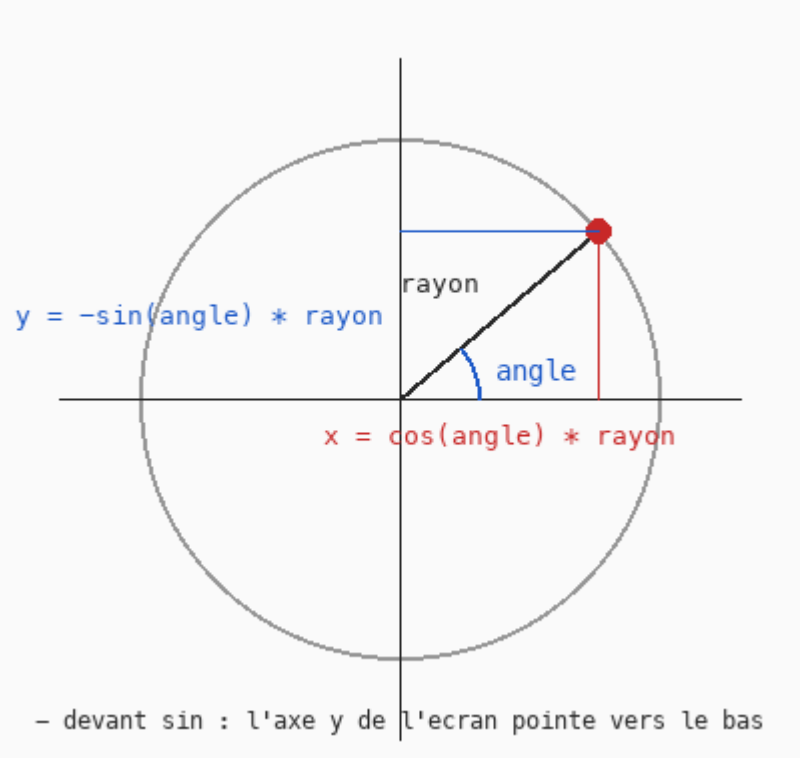
1. Étape 1 — placer un point avec un angle

Dans le `.h` : `int nbCercles { 12 };` et `float rayonOrbite { 200 };`. Fenêtre 800 × 800, et dans `draw()` :

```
        ofBackground(0);
        ofFill();
        ofSetColor(255);

        for (int i = 0; i < nbCercles; i++) {
                float angle = i * TWO_PI / nbCercles;
                float x = 400 + cos(angle) * rayonOrbite;
                float y = 400 - sin(angle) * rayonOrbite;
                ofDrawCircle(x, y, 40);
        }
```

Douze cercles blancs, régulièrement répartis sur un anneau. La formule à retenir pour toute la suite :



- `cos(a)` est la coordonnée **horizontale** d'un point sur le cercle de rayon 1 à l'angle `a` ; `sin(a)` sa coordonnée **verticale**. On multiplie par le rayon voulu, et on ajoute le centre.
- Le signe `-` devant `sin` vient du repère de l'écran (cours 00) : `y` pointe vers le bas ; sans le `-`, les angles tourneraient dans le sens des aiguilles d'une montre.
- Les angles sont en **radians** : un tour complet vaut `2π` (~6.28). openFrameworks fournit `PI` et `TWO_PI` — et `ofDegToRad(90)` si tu préfères penser en degrés. Pour répartir `n` cercles : le numéro `i` est à l'angle `i * TWO_PI / n`.

Au cours 07, `cos(t)` faisait **osciller** un nombre. C'est le même `cos` : quand l'angle avance, le point tourne — et sa coordonnée `x` oscille.

> **Essaie** : `nbCercles` à 3, 7, puis 36. Puis change le rayon des petits cercles.

2. Étape 2 — déplacer l'origine

Les `400 +` partout sont pénibles. Mieux : déplacer l'**origine** au centre.

```
ofPushMatrix();
ofTranslate(400, 400);

for (int i = 0; i < nbCercles; i++) {
        float angle = i * TWO_PI / nbCercles;
        float x = cos(angle) * rayonOrbite;
        float y = -sin(angle) * rayonOrbite;
        ofDrawCircle(x, y, 40);
}

ofPopMatrix();
```

`ofTranslate(400, 400)` : tout ce qui est dessiné ensuite a son `(0, 0)` au centre de la fenêtre. `ofPushMatrix` mémorise l'origine actuelle, `ofPopMatrix` la restaure — entre les deux, tout est décalé ; après, on redessine normalement. **Toujours par paire.**

> **Essaie** : `ofTranslate(mouseX, mouseY)` — tout l'anneau suit la souris, sans toucher aux formules.

3. Étape 3 — `%` : le reste qui fait alterner

Colore les cercles en alternance :

```
        if (i % 3 == 0)      ofSetColor(255, 0, 0);
        else if (i % 3 == 1) ofSetColor(0, 255, 0);
        else                 ofSetColor(0, 0, 255);
```

`i % 3` est le **reste** de la division entière de `i` par 3 : 0, 1, 2, 0, 1, 2, … C'est l'outil pour alterner, cycler, répartir en groupes. `else if` enchaîne les conditions — la première vraie gagne, `else` attrape le reste.

`%` ne marche que sur des **entiers** ; pour les `float`, c'est `fmod` (cours 09).

> **Essaie** : la roue des teintes du cours 09 **est** un cercle — remplace l'alternance par `ofSetColor(ofColor::fromHsb(i * 255.0f / nbCercles, 255, 255));` : chaque cercle prend la couleur de son secteur. Deux idées de deux cours différents qui étaient la même.

4. Étape 4 — les modes de fusion

Encadre le dessin :

```
ofEnableBlendMode(OF_BLENDMODE_ADD);
// ... la boucle de dessin
ofEnableBlendMode(OF_BLENDMODE_ALPHA);   // retour au mode normal
```

Par défaut, une forme recouvre ce qu'il y a dessous (en tenant compte de la transparence). En mode **additif**, les lumières s'**ajoutent** : rouge sur vert donne jaune, les trois donnent blanc — le cube RGB du cours 09, rendu visible aux intersections. Autres modes : `SUBTRACT`, `MULTIPLY`, `SCREEN`. On revient toujours à `ALPHA` après.

> **Essaie** : fond blanc et `OF_BLENDMODE_SUBTRACT` — le négatif du mélange.

5. Étape 5 — l'animation : l'anneau qui respire

Dans le `.h` : `float t { 1 };`. Dans `update()` :

```
t = cos(ofGetElapsedTimef() * 0.6f);
```

Et dans les formules : `cos(angle) * rayonOrbite * t` (pareil pour `y`). `t` oscille entre −1 et 1 (cours 07) : tous les cercles se rapprochent du centre, **traversent** de l'autre côté quand `t` devient négatif, et repartent. Une seule variable de temps, douze cercles qui respirent ensemble.

> **Essaie** : fais aussi **tourner** l'ensemble — ajoute `ofGetElapsedTimef()` à `angle`.

Exercices

1. **Lire avant de lancer** — avec `nbCercles = 4` et le `- sin`, où sont les quatre cercles : haut / bas / gauche / droite, ou dans les coins ? Et le cercle `i = 1`, où exactement ? Réponds, puis vérifie.
2. **Les deux anneaux** — un second `for` : autre rayon d'orbite, autre nombre de cercles, et un sens de rotation **inverse** (soustrais le temps au lieu de l'ajouter).
3. **L'horloge** — une aiguille des secondes : un trait (`ofDrawLine`) du centre vers un point sur le cercle, qui fait exactement **un tour en 60 secondes**. Quelle formule d'angle ? Bonus : le sens horaire est l'inverse du sens trigonométrique.
4. **Le cercle cassé** *(plus costaud)* — la variante du sketch d'origine : 120 petits cercles (rayon 10), fond blanc, mode `SUBTRACT`, et un décalage qui casse le cercle parfait : `x += cos(i * 2.0f) * 20; y += sin(i / 20.0f) * 100;`. Observe comme deux petites perturbations suffisent à faire naître une forme organique.

Ce qu'il faut retenir

- Un point à l'angle `a`, à la distance `R` : `(cos(a) * R, -sin(a) * R)` — le `-` à cause du `y` vers le bas.
- Les angles sont en **radians** : un tour = `TWO_PI` ; répartir `n` objets = `i * TWO_PI / n`.
- `ofTranslate` déplace l'origine ; `ofPushMatrix` / `ofPopMatrix` l'encadrent, toujours par paire.
- `i % n` : le reste — alterner, cycler, grouper (entiers seulement, `fmod` pour les `float`).
- `OF_BLENDMODE_ADD` : les lumières s'additionnent ; revenir à `ALPHA` après.
- Animer un rayon par un `cos(temps)` : tout l'anneau respire d'une seule variable.